

jcode & OpenRouter Installation Guide

A complete technical workflow for deploying a multi-agent terminal harness.

1. Core Installation

jcode acts as an orchestration layer. To get started, execute the appropriate shell command for your environment to pull and install the core binaries.

For Windows 11 (PowerShell 5.1+)

```
irm https://jcode.sh/install.ps1 | iex
```

For macOS & Linux (Bash/Zsh)

```
curl -fsSL https://jcode.sh/install | bash
```

2. OpenRouter Authentication

OpenRouter provides access to a vast array of models (including free ones) through a single API gateway. You will need an API key from openrouter.ai/keys.

Once you have your key, run the built-in authentication flow:

```
jcode login --provider openrouter
```

When prompted, paste your API key. jcode will securely encrypt and store this locally.

Setting a Default Model

To avoid typing the provider and model flags on every invocation, you can set a default OpenRouter endpoint. (Replace `YOUR_API_KEY` with your actual key).

```
printf '%s' "YOUR_API_KEY" | jcode provider add openrouter-default --base-url https://openrou
```

3. The Extended Tool & Skill Ecosystem

jcode is designed to be highly modular. By chaining MCPs (Model Context Protocol servers) and standalone tools, you can give the agent browser automation, local task tracking, and specialized file manipulation.

- `agentgrep` : Semantic, AST-aware code search optimization.
- `firefox-agent-bridge` : Grants DOM reading and automated UI testing capabilities.
- `mermaid-rs-renderer` : Native Rust-based architecture diagram generation.
- `sksync` & `mikan` : Skill synchronizers and local micro-kanban boards for autonomous multi-step execution.
- `ECC` & `ruflo` : Claude Code toolkits adapted for advanced file system operations and cross-harness compatibility.

4. The Master Setup Prompt

Once jcode is installed and authenticated, you can ask the AI to bootstrap its own environment. Start a session by typing `jcode run` (or start an interactive chat), and copy-paste the exact prompt below. The agent will orchestrate the cloning, compilation, and configuration of all necessary dependencies.

Initialize a comprehensive local agent workflow environment. I need you to orchestrate the installation and configuration of several modular skills and tools designed for jcode and Claude Code interoperability. Execute the necessary terminal commands to:

- 1. Clone and compile 1jehuang/agentgrep for semantic AST-aware code searches.*
- 2. Install 1jehuang/firefox-agent-bridge to enable browser automation capabilities.*
- 3. Setup 1jehuang/mermaid-rs-renderer for fast, native diagram rendering in the terminal.*
- 4. Install takemo101/sksync to manage and synchronize our agent skill definitions.*
- 5. Pull takemo101/mikan and initialize a local micro-kanban board for multi-step coding tasks.*
- 6. Pull down the affaan-m/ECC (Everyday Claude Code) toolkit and adapt its bash and file-system MCPs for use within this jcode environment.*
- 7. Locate and install ruflo alongside any other standard Claude-compatible toolchains to expand your file manipulation and terminal-execution capabilities.*

Proceed step-by-step. Verify the success of each repository clone and installation script before moving to the next, and update my local configuration files to register these tools permanently so they are available in future sessions.